

Worksheet 1: Solutions

Introduction to Quantum Circuits — Introduction to Quantum Computation

1.

- (a) The gate leaves a and b alone and sends $c \mapsto c \oplus ab$:

a	b	c	a'	b'	c'
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

Only the last two rows are altered: the gate swaps $110 \leftrightarrow 111$ and fixes everything else.

- (b) The eight outputs are a permutation of the eight inputs — no output appears twice — so the map is invertible. Applying it again sends $c \mapsto (c \oplus ab) \oplus ab = c$, since $ab \oplus ab = 0$. The Toffoli is its own inverse.
- (c) A reversible gate must have as many outputs as inputs, and must not send two different inputs to the same output. AND fails on both counts: $(0,0)$, $(0,1)$ and $(1,0)$ all give 0, so from the output alone the input cannot be recovered — information is destroyed. Writing the answer onto a fresh qubit keeps the inputs around, so nothing is lost and the gate can be reversible. That spare qubit is the price of reversibility.

Setting $c = 0$ in the table of part (a) leaves $c' = 0 \oplus ab = ab$, so $\text{ccx}(0,1,2)$ with qubit 2 starting in $|0\rangle$ is an AND gate — one that happens to keep its inputs. This is the building block of Q2 and Q4: every gate there is built by negating inputs or outputs around one or more Toffolis.

2.

- (a) $\text{NOR}(a,b) = \bar{a} \wedge \bar{b}$. Negate both inputs, then AND them onto qubit 2:

$$\mathbf{x}(0), \quad \mathbf{x}(1), \quad \text{ccx}(0,1,2)$$

Qubit 2 is flipped only when both $\bar{a}$ and $\bar{b}$ are 1, i.e. only when $a = b = 0$. ✓

- (b) $a \Rightarrow b = \overline{a \wedge \bar{b}}$. Negate b , AND onto qubit 2, then negate the result:

$$\mathbf{x}(1), \quad \text{ccx}(0,1,2), \quad \mathbf{x}(2)$$

The Toffoli fires only for $a = 1, b = 0$, so qubit 2 is 1 in that case and 0 otherwise; the final $\mathbf{x}(2)$ inverts this to give 0 for $a = 1, b = 0$ and 1 elsewhere. ✓

(More than one circuit is correct here. If you negated qubit 0 and built $\bar{a} \vee b$ instead, you should have arrived at something of the same length.)

3.

- (a) $\text{cx}(0,1), \quad \mathbf{x}(1)$. The cx leaves qubit 1 holding $a \oplus b$, which is XOR; the $\mathbf{x}$ negates it to give XNOR, which is 1 exactly when $a = b$. ✓

- (b) The map $(a, b) \mapsto (a, a \oplus b)$ is a bijection on two bits: from the output pair the input is recovered as $(a, a \oplus (a \oplus b))$. Nothing is destroyed, so it can be done in place with no spare qubit. The map $(a, b) \mapsto a \wedge b$ is three-to-one on its 0 output and so cannot be. The rule of thumb: **a gate needs an ancilla exactly when it loses information.**

4.

- (a) The three Toffolis fire on the pairs $(0, 1)$, $(1, 2)$ and $(0, 2)$ respectively, so the number that fire is the number of *pairs* of inputs that are both 1. If k of the three inputs are 1, that is $\binom{k}{2}$:

k (inputs equal to 1)	$\binom{k}{2}$ flips	parity	output
0	0	even	0
1	0	even	0
2	1	odd	1
3	3	odd	1

The output is 1 exactly when $k \geq 2$. ✓ The $k = 3$ case is the pretty one: three flips, not one, but three is odd so the answer still comes out right.

- (b) Qubit 3 starts at 0 and each Toffoli that fires flips it. Flipping twice restores it, so only the parity of the number of flips survives — the count itself is irrelevant. This is the same $\mathbb{Z}_2$ arithmetic as $c \mapsto c \oplus ab$ in Q1.

5.

- (a) The graph is a “T” (or “Y”): qubit 1 joined to 0, 2 and 3, with 4 hanging off 3.

qubit	0	1	2	3	4
neighbours	1	3	1	2	1

Qubit 1 is the only one with three neighbours.

- (b) Put the **control on qubit 1** and the three targets on 0, 2 and 3 in any order. Every **cx** is then between directly connected qubits, so no **swap** is needed and the cost is just **3 cx gates**.
- (c) Qubit 3 has only two neighbours, 1 and 4, so two targets can be reached directly but the third cannot — at least one **swap** is forced. Best arrangement: targets on 1 and 4, the third on 0 (or 2). Do **cx(3, 1)** and **cx(3, 4)** first, then **swap(0, 1)** to bring the last target next to the control, then **cx(3, 1)**. Total $3 + 3 = 6$ **cx gates** — twice the cost of (b), from a single bad placement.
- (d) No: four targets plus a control is five qubits, and the control would need four neighbours, while the best on this device is three. On the line, no qubit has more than two neighbours, so even three targets force at least one **swap**; the best is $3 + 3 = 6$ **cx gates**, with the control in the middle.